

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 35%

QUESTIONS: 5

Duration : 1 Hour

Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I ___N.A.Sriharan/ 1911260036___ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug ont

the ground, water can be poured from one jug to another and that there are no other measuring devices available.

ANSWER:

Let:

- **A = 4-gallon jug**
- **B = 3-gallon jug**
- Initial state = **(0, 0)**

Step	Action	4-gallon A	3-gallon B
1	Fill the 3-gallon jug	0	3
2	Pour B → A	3	0
3	Fill the 3-gallon jug again	3	3
4	Pour B → A until A is full	4	2
5	Empty the 4-gallon jug	0	2
6	Pour the 2 gallons from B → A	2	0

Final state:

4-gallon jug = 2 gallons

3-gallon jug = 0 gallons

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.

- What are the percept's for this agent?
- Characterize the operating environment.
- What are the actions the agent can take?
- How can one evaluate the performance of the agent?
- What sort of agent architecture do you think is most suitable for this agent? Why?

ANSWER:

2. Mars Rover – PEAS Analysis

The Mars Rover is an autonomous agent that explores the Martian surface, and collects and analyses soil/rock samples. It is described using the PEAS framework:

i. Percepts

- Camera images (stereo/panoramic) of terrain, rocks and obstacles
- Sensor readings: wheel odometry, tilt/inclination, accelerometer, gyroscope, wheel-motor current
- Spectrometer, X-ray and chemical sensor data from soil and rock samples
- Temperature, wind, dust and radiation measurements; battery/solar power level
- GPS-like localisation/navigation data, and commands received from Earth (via orbiter relay)

ii. Operating Environment Property

Characterisation

Observability	Partially observable – sensors only see the local surroundings
Agents	Single agent (no other competing agents)
Determinism	Stochastic – wheel slip, dust storms and uneven terrain make outcomes uncertain
Episodic / Sequential	Sequential – current moves affect future ones (power, position)
Static / Dynamic	Dynamic – weather, dust, lighting and temperature change while the rover acts
Discrete / Continuous	Continuous – positions, speeds, temperatures and time are continuous
Known / Unknown	Partly unknown – terrain and samples are new and unexplored

iii. Actions (Actuators)

- Move forward/backward, turn, steer and stop the wheels; climb or avoid obstacles
- Move the robotic arm; drill, scoop or grind rock and soil
- Deploy and point cameras and instruments; place samples in the on-board analysis lab
- Transmit data/images to Earth; receive commands
- Manage power: orient solar panels, switch to sleep/low-power mode, heat the electronics

iv. Performance Measure

- Number and scientific value of samples collected and analysed correctly
- Area explored / distance covered safely
- Safety: avoiding falls, getting stuck or damage; rover survival/lifetime
- Efficient use of energy, time and communication bandwidth
- Quality and amount of useful data successfully sent back to Earth

v. Suitable Agent Architecture

A utility-based agent with learning capability (a hybrid with reactive layers) is the most suitable. Reasons:

- Communication delay (4–24 minutes one way) means the rover cannot rely on Earth for real-time control – it must decide autonomously.
- A simple reflex agent is not enough, because the environment is partially observable and the rover needs to remember maps and past samples (model-based).
- It has multiple, sometimes conflicting goals (explore more vs. save power vs. stay safe vs. collect valuable samples). A utility function lets it trade these off, which a plain goal-based agent cannot do.
- A reactive layer handles emergencies instantly (e.g. stop on detecting a cliff), while a learning element improves terrain classification and navigation over time in an unknown environment.

3. 3. 8-Queens Problem – Search Space and Formulation

Goal: place 8 queens on an 8×8 chessboard so that no two queens attack each other, i.e. no two share the same row, column or diagonal. The configuration given in the figure is not a solution because the queen in the first column and the queen in the last column lie on the same diagonal.

A. Problem formulation – Incremental approach (preferred) Component

Definition

States

Any arrangement of 0 to 8 queens on the board. (Better version: one queen per column in the leftmost n columns, with no queen attacking another.)

Initial state

Empty board – no queens placed

Actions

Add a queen to a square in the leftmost empty column such that it is not attacked by

	any queen already on the board
Transition model	Returns the board with the queen added to the specified square
Goal test	8 queens are on the board and none is attacked
Path cost	Not relevant – only the final configuration matters (each move can be given cost 1)

B. Alternative – Complete-state formulation

- States: all 8 queens on the board (one per column); the state can be written as a list $[r_1, \dots, r_8]$ where r_i is the row of the queen in column i .
- Initial state: any random arrangement of 8 queens (e.g. the figure's configuration).
- Actions: move one queen to another square in the same column.
- Goal test: no pair of queens attacks each other (number of attacking pairs $h = 0$).

C. Size of the search space Formulation	Number of states
Any 8 queens on any squares ($64C8$)	$\approx 4.4 \times 10^9$ (1.8×10^{14} sequences $64 \times 63 \times \dots \times 57$)
One queen per column, any row (8^8)	16,777,216
One queen per row and column ($8!$ permutations)	40,320
Incremental, only non-attacking placements	2,057 states

4. OLA Cab Booking – Problem-Solving Agent

Type of agent: A **goal-based agent** (with utility-based selection of the cab type). The goal is to reach the destination; the customer's comfort/cost preferences (mini, micro, sedan, shared, prime...) act as a utility function that picks the best cab among the options that achieve the goal. It is a problem-solving agent that formulates the goal, finds options and chooses the best.

Problem formulation

- Initial state: customer at the pickup location, no cab booked.

- Goal state: customer dropped at the destination.
- Actions: enter locations, view cab types, select a cab type, confirm booking, track the cab, start the trip, end the trip, pay.
- Performance measure: low fare, short waiting/travel time, comfort and safety.

Pseudocode

```

function OLA-CAB-AGENT(percept) returns action
persistent: state, goal, options, preference

state <- UPDATE-STATE(state, percept) // pickup, drop, time, traffic, cab availability
if goal is empty then
goal <- FORMULATE-GOAL(state) // reach destination D from pickup P
preference <- GET-USER-PREFERENCE() // budget, comfort, time, shared/private
if state.pickup = empty or state.drop = empty then
return ASK-LOCATIONS()

options <- GET-AVAILABLE-CABS(state.pickup, state.drop) // mini, micro, sedan, shared,
prime

if options is empty then return NOTIFY("No cab available")

best <- NULL; bestUtility <- -infinity

for each cab in options do
fare <- CALCULATE-FARE(cab, distance, time)
eta <- ESTIMATE-ARRIVAL(cab)
u <- UTILITY(cab, fare, eta, comfort(cab), preference)

if u > bestUtility then
best <- cab; bestUtility <- u

selected <- USER-CONFIRM(best, options) // user may override with another type
booking <- BOOK-CAB(selected)

if booking.status = FAILED then return RETRY(options \ selected)
TRACK-CAB(booking) // driver details, live location

if state.onBoard = true then NAVIGATE(state.pickup -> state.drop)

if state.location = state.drop then

```

COMPLETE-TRIP(); PAYMENT(fare); GET-FEEDBACK()

goal <- empty

return action

5. Uniform Cost Search (UCS) – Least-Cost Path from S to G

Edges read from the graph From → To	Cost	Remark
S → A	1	
S → G	12	Direct route
A → B	3	
A → C	1	
B → D	3	
C → D	1	
C → G	2	
D → G	3	

UCS keeps a priority queue (frontier) ordered by path cost $g(n)$ and always expands the cheapest node first. The goal test is applied when a node is selected for expansion.

Step-by-step execution Step	Expand	Add / update in frontier	Frontier after (node:cost)
1	S (0)	A = 1, G = 12	A:1, G:12
2	A (1)	B = 1+3 = 4, C = 1+1 = 2	C:2, B:4, G:12
3	C (2)	D = 2+1 = 3, G = 2+2 = 4 (better than 12, so update)	D:3, B:4, G:4
4	D (3)	G via D = 3+3 = 6 (worse than 4, ignore)	B:4, G:4
5	B (4)	D via B = 4+3 = 7 (worse than 3, ignore)	G:4

6

G (4)

Goal test succeeds –
– stop

Order of expansion: $S \rightarrow A \rightarrow C \rightarrow D \rightarrow B \rightarrow G$

Least-cost path: $S \rightarrow A \rightarrow C \rightarrow G$ with total cost = $1 + 1 + 2 = 4$

Comparison of other paths	Cost	Optimal?
$S \rightarrow A \rightarrow C \rightarrow G$	4	Yes
$S \rightarrow A \rightarrow C \rightarrow D \rightarrow G$	$1+1+1+3 = 6$	No
Path	Cost	Optimal?
$S \rightarrow A \rightarrow B \rightarrow D \rightarrow G$	$1+3+3+3 = 10$	No
$S \rightarrow G$	12	No

Conclusion: The company should route packages along $S \rightarrow A \rightarrow C \rightarrow G$, which costs 4 (fuel + time). UCS is complete and optimal because all edge costs are positive